Smart Parking Fee Management System

Problem statement: Design a Smart Parking Fee Management System that calculates parking charges based on vehicle category, entry time, and exit time.

The system records vehicle details, calculates the total parking duration, applies the correct fee according to the vehicle category, and displays the final amount.

1. Computational Thinking

Computational thinking involves solving a problem using logical and systematic steps.

The four main concepts are:

1. Decomposition: Divide the parking system into smaller tasks:

- Enter vehicle number.
- Select vehicle category.
- Record entry time.
- Record exit time.
- Calculate parking duration.
- Calculate parking fee.
- Display the bill.

2. Pattern Recognition: Identify common patterns, such as different vehicles being charged according to their category and parking duration.

3. Abstraction: Focus only on important information, such as vehicle type, entry time, exit time, and fee.

4. Algorithm Design: Develop step-by-step instructions to calculate parking charges.

2. Variables

Variables store the information used in the program.

Variable	Purpose
vehicle_no	Vehicle registration number
vehicle_type	Type of vehicle
entry_time	Parking entry time
exit_time	Parking exit time

total_hours	Total parking duration
fee	Parking charge
total_fee	Final amount

3. Data Types

Data type	Example	Usage
Integer	5	Total parking hours
Float	50.5	Parking fee
String	"AP39AB1234"	Vehicle number
Boolean	True / False	Valid time check
List	[20, 30, 50]	Parking fee rates
Tuple	("Car", 30)	Vehicle category and rate

4. Operators

Operators perform calculations and comparisons.

Operator	Meaning	Example
+	Addition	fee + 10
-	Subtraction	exit - entry
*	Multiplication	hours * rate
/	Division	minutes / 60
%	Modulus	minutes % 60
==	Equal to	type == "Car"
!=	Not equal	type != "Bike"
>	Greater than	hours > 2
<	Less than	hours < 5
and	Logical AND	hours > 0 and hours < 5
or	Logical OR	type == "Bike" or type == "Car"

5. Conditional Statements

Conditional statements determine parking charges based on vehicle category.

Assume the following parking rates:

Vehicle category	Fee per hour
Bike	₹10
Car	₹20
Bus	₹50
Truck	₹40

Example:

python

```
if vehicle_type == "Bike":
    rate = 10
elif vehicle_type == "Car":
    rate = 20
elif vehicle_type == "Bus":
    rate = 50
elif vehicle_type == "Truck":
    rate = 40
else:
    print("Invalid vehicle type")
```

6. Looping Statements

Loops are used to repeat instructions.

For loop: Display all vehicle categories.

python

```
vehicles = ["Bike", "Car", "Bus", "Truck"]

for vehicle in vehicles:
    print(vehicle)
```

While loop: Continue accepting vehicle details until the user chooses to stop.

python

```
choice = "yes"


while choice == "yes":
    print("Enter vehicle details")
    choice = input("Continue? yes/no: ")
```

7. String Pattern Generation

Strings are used to store and display vehicle numbers and parking bills.

Example:

python

```
vehicle_no = "AP39AB1234"


print("*" * 30)
print("SMART PARKING SYSTEM")
print("*" * 30)
print("Vehicle:", vehicle_no)
print("*" * 30)
```

Output:

text

```
******************************
SMART PARKING SYSTEM
******************************
Vehicle: AP39AB1234
******************************
```

8. Lists

A list stores multiple vehicle categories or parking fees.

python

```
vehicles = ["Bike", "Car", "Bus", "Truck"]
```

```
rates = [10, 20, 50, 40]


print(vehicles)
print(rates)
```

9. Tuples and Packing

A tuple stores multiple values in a single variable. Tuples are immutable.

Tuple example:

python

```
bike = ("Bike", 10)
car = ("Car", 20)
bus = ("Bus", 50)
truck = ("Truck", 40)


print(bike)
```

Packing:

python

```
vehicle_details = ("AP39AB1234", "Car", 9, 12)
```

Unpacking:

python

```
vehicle_no, vehicle_type, entry, exit = vehicle_details


print(vehicle_no)
print(vehicle_type)
print(entry)
print(exit)
```

10. Algorithm

Step-by-step procedure:

1. Start the program.
2. Enter vehicle registration number.

3. Select vehicle category.
4. Enter vehicle entry time.
5. Enter vehicle exit time.
6. Calculate total parking hours.
7. Check vehicle category.
8. Assign parking rate.
9. Calculate parking fee.
10. Display vehicle details and total fee.
11. Stop the program.

11. Flowchart

id.scribd.com

Flowchart steps in text:

text

```
      START
        |
        v
  Enter Vehicle Details
        |
        v
  Enter Entry and Exit Time
        |
        v
  Calculate Parking Hours
        |
        v
  Check Vehicle Category
        |
        v
```

Assign Parking Rate

|

v

Calculate Total Fee

|

v

Display Parking Bill

|

v

STOP

12. Pseudocode

text

```
START

INPUT vehicle_no
INPUT vehicle_type
INPUT entry_time
INPUT exit_time

IF exit_time < entry_time THEN
    PRINT "Invalid Time"
ELSE
    total_hours = exit_time - entry_time

    IF vehicle_type = "Bike" THEN
        rate = 10
    ELSE IF vehicle_type = "Car" THEN
        rate = 20
    ELSE IF vehicle_type = "Bus" THEN
```

```
        rate = 50
    ELSE IF vehicle_type = "Truck" THEN
        rate = 40
    ELSE
        PRINT "Invalid Vehicle Type"
    END IF

    fee = total_hours * rate

    PRINT "Vehicle Number:", vehicle_no
    PRINT "Vehicle Type:", vehicle_type
    PRINT "Parking Hours:", total_hours
    PRINT "Total Parking Fee:", fee
END IF

STOP
```

13. Python Program

Here is a complete basic implementation:

python

```
print("===== SMART PARKING FEE MANAGEMENT =====")

vehicle_no = input("Enter vehicle number: ")
vehicle_type = input("Enter vehicle type (Bike/Car/Bus/Truck): ")

entry_time = int(input("Enter entry hour (0-23): "))
exit_time = int(input("Enter exit hour (0-23): "))

if not (0 <= entry_time <= 23 and 0 <= exit_time <= 23):
```

```
    print("Invalid time")

elif exit_time < entry_time:
    print("Invalid exit time")

else:
    total_hours = exit_time - entry_time

    rates = {
        "Bike": 10,
        "Car": 20,
        "Bus": 50,
        "Truck": 40
    }

    if vehicle_type in rates:
        rate = rates[vehicle_type]

        # Minimum one-hour charge
        if total_hours == 0:
            total_hours = 1

        fee = total_hours * rate

        print("\n======= PARKING BILL =======")
        print("Vehicle Number:", vehicle_no)
        print("Vehicle Type:", vehicle_type)
        print("Parking Hours:", total_hours)
        print("Rate per Hour: Rs.", rate)
```

```
        print("Total Fee: Rs.", fee)
        print("============================")

    else:
        print("Invalid vehicle category")
```

14. Sample Output

text

```
===== SMART PARKING FEE MANAGEMENT =====

Enter vehicle number: AP39AB1234
Enter vehicle type: Car
Enter entry hour: 9
Enter exit hour: 12

======= PARKING BILL =======
Vehicle Number: AP39AB1234
Vehicle Type: Car
Parking Hours: 3
Rate per Hour: Rs. 20
Total Fee: Rs. 60
============================
```

15. Expected Result

The Smart Parking Fee Management System successfully calculates parking charges based on vehicle category and parking duration using:

- Computational thinking
- Variables and data types
- Arithmetic and logical operators
- Conditional statements
- For and while loops

- String pattern generation
- Lists and tuples
- Packing and unpacking
- Algorithms and flowcharts
- Pseudocode and Python programming

Conclusion: This system reduces manual calculation, saves time, and provides accurate parking fee management for different vehicle categories.